Programación Orientada a Objetos en C++: Diseño e Implementación de un Simulador de Combate Espacial en Consola (*class Nave*)

Malek Cheheid

E.E.S.T. N.° 1 "Eduardo Ader" — Vicente López

LPR: Laboratorio de Programación

5.° año, 3.°, grupo A — 2.° cuatrimestre

Actividad N.° 8: Programación Orientada a Objetos (POO) en C++

Profesor de grupo: Yamil Ganduglia

21 de octubre de 2026

Repositorios del proyecto

GitHub: https://github.com/malekcheheid-art/juegodenaves.git

GitLab: https://gitlab.com/malekcheheid-art/juegodenaves.git

Introducción

Para la Actividad N.° 8 de Laboratorio de Programación (LPR) trabajé sobre un simulador de combate espacial en consola, escrito en C++. El profesor me entregó el código base en `src/main.cpp`; a mí me tocó compilarlo, leerlo línea por línea, explicar sus decisiones de diseño y sumarle una funcionalidad propia. La actividad me pedía además dar el paso del `struct` —que usé en la Actividad N.° 7 para agrupar variables sueltas— a una `class`, que además de agrupar los datos los guarda adentro y decide desde afuera qué se puede tocar. Ese es el punto central de la Programación Orientada a Objetos (cplusplus.com, s. f.).

El juego entero vive en ese único archivo, con 372 líneas. Ahí hay tres clases: `Nave`, que es el jugador; `Asteroide`, que cae desde arriba y vuelve a aparecer en una columna aleatoria cuando llega abajo; y `Bala`, que sube desde la nave y desaparece al salir de la pantalla. El bucle principal de `main()` lee el teclado sin frenar el juego, usando `_kbhit()` y `_getch()`, mueve las balas, hace descender los asteroides y compara posiciones para detectar los impactos. Ese motor lo compilé con TDM-GCC 10.3 en modo C++17 y con las advertencias activadas (`-Wall -Wextra`), y terminó sin errores.

Trabajé siguiendo la metodología 5R del profesor: **registrar** (copié el código, lo compilé y lo ejecuté), **reducir** (marqué dónde termina la sección `private:` y dónde empieza la `public:`), **responder** (este cuestionario técnico), **reflexionar** (agregué una funcionalidad extra) y **revisar** (publiqué todo en los dos repositorios).

Como desafío obligatorio para aprobar, implementé una sola funcionalidad de las 15 del anexo: la N.° 1, el marcador de puntos o *score*.

Desarrollo

Qué Diferencia a un `struct` de una `class`. En C++, las dos palabras hacen casi lo mismo: las dos agrupan atributos y métodos, las dos aceptan constructores, sobrecarga y herencia. La única diferencia de regla es el acceso por defecto. Todo lo que se declara con `struct` es `public` por defecto; todo lo que se declara con `class` es `private` por defecto. La herencia sigue la misma lógica: pública en `struct`, privada en `class` (cplusplus.com, s. f.; GeeksforGeeks, s. f.).

En la práctica eso se nota en la clase `Nave` que trae el código base. Está en `src/main.cpp`, líneas 84 a 161, y tiene esta forma:

```
class Nave {
private:
    int x, y;     // Coordenadas de la nave en la consola
    int vidas;    // Cantidad de vidas restantes

public:
    Nave(int _x, int _y, int _vidas) { ... }
    int getX() { return x; }
    void reducirVida() { vidas--; }
    ...
};
```

Los tres atributos —`x`, `y` y `vidas`— quedan antes de cualquier especificador de acceso, así que están privados. Por consecuencia, si alguien escribiera `jugador.vidas = 99;` dentro de `main()`, el compilador frenaría el proceso con un error de acceso. Si esa misma clase se hubiera declarado con la palabra `struct` en lugar de `class`, la línea compilaría y correría sin avisar nada. Ahí está la diferencia real: con `struct` se deja la puerta abierta y con `class` se la cierra. Por eso digo que el `struct` junta datos sueltos, y que la `class` arma una caja fuerte que contiene los datos y, además, las funciones que tienen derecho a moverlos.

Por Qué No Dejé la *Salud* como Pública. Si `vidas` fuera `public`, cualquiera podría escribirle cualquier cosa. Basta con que un programador novato escriba, por descuido, `jugador.vidas = -5;`: el programa compila igual. Después, dentro del bucle, la condición `if (jugador.getVidas() == 0)` ya no se cumple nunca,

`gameOver` nunca pasa a `true` y el juego queda girando en un bucle infinito mientras el HUD anuncia vidas negativas. Si en cambio se escribe `jugador.vidas = 999999;`, desaparece el desafío y la partida pierde todo el sentido. En los dos casos el error es silencioso: no lo ve el compilador, lo ve —o no ve— el que está jugando.

Por eso el código encapsula `vidas` en `private` y deja dos puertas nomás. Una de lectura, `getVidas()` en la línea 112, que devuelve el valor pero no permite escribirlo. Y una de escritura, `reducirVida()` en la línea 115, cuyo cuerpo entero es `vidas--;`. Como existe una sola vía para cambiarla y esa vía únicamente resta de a uno, el estado de la nave no se puede descomponer. Con este diseño el objeto cuida su propio estado sin que yo tenga que estar pendiente, y si algún día quiero agregarle un escudo o una regeneración, cambio la clase y no tengo que tocar una sola línea de `main()`.

Para Qué Sirve el Constructor de `Nave`. El constructor es la función miembro que se ejecuta sola, una única vez, en el instante exacto en que se crea el objeto. Se llama con el mismo nombre que la clase y no devuelve nada, ni siquiera `void` (Microsoft, 2022; cplusplus.com, s. f.). En el código base está en la línea 103 y tiene esta forma:

```
Nave(int _x, int _y, int _vidas) {
    x = _x;
    y = _y;
    vidas = _vidas;
}
```

Se invoca en `main()` con la sentencia `Nave jugador(35, 20, 3);`, en la línea 249.

El trabajo del constructor es dejar el objeto en un estado válido antes de que cualquier método lo toque. Si se borra ese bloque, al declarar `jugador` las variables `x`, `y` y `vidas` no reciben ningún valor. Como son tipos básicos, la memoria queda con basura y las primeras llamadas a `getVidas()` o a `mover()` devuelven y operan con números indeterminados: eso en C++ se llama comportamiento indefinido. El guion bajo de los parámetros (`_x`, `_y`, `_vidas`) está ahí para distinguirlos de los atributos, porque los dos se llaman igual y dentro del constructor la palabra `x` se resuelve hacia el miembro de la clase. Existe además una forma más prolija, la lista de inicialización de miembros, que asigna

antes de entrar al cuerpo del constructor: `Nave(int x, int y, int vidas) : x(x), y(y), vidas(vidas) {}` (Microsoft, 2022).

Aclaro una diferencia con el enunciado. La pregunta del cuestionario menciona un bloque `Nave(string _nombre, ...)`. En el código que entregó el profesor el nombre del piloto no se guarda dentro de la clase: se lee en `main()` con `getline(cin, nombrePiloto)` antes de arrancar la partida, se usa como sistema anti-copia y se dibuja directamente en la cabecera de la consola, en la línea 274. Me parece que está bien resuelto así. El nombre es un dato para mostrar en pantalla, no un estado de la nave; si estuviera dentro de la clase, la tendrían que obligar a saber dibujar y se mezclarían dos responsabilidades que conviene separar. Si decidiera sumarlo, tendría que cambiar la firma a `Nave(string _nombre, int _x, int _y, int _vidas)`, declarar el atributo `string nombre` y agregar su accesor `getNombre()`.

Qué Funcionalidad Agregué y Dónde la Inserté. Elegí la funcionalidad N.° 1, el marcador de puntos. La implementé con tres toques puntuales dentro de `src/main.cpp`, sin modificar ninguna de las tres clases que ya traía el motor.

Primero, declaré el acumulador en la línea 246, al lado de `bool gameOver = false;`: `int score = 0;`. Lo dejé como variable local de `main()`, no como variable global. Prefiero que viva en ese ámbito: una global se puede cambiar desde cualquier punto del archivo y eso rompe justo lo que estoy aprendiendo a hacer en esta actividad.

Segundo, lo muestro en el HUD, dentro del bucle `while (!gameOver)`, en las líneas 279 y 280:

```
gotoxy(15, 2);
cout << "SCORE: " << score << "  ";
```

Los dos espacios del final no son de adorno: sirven para borrar los dígitos que sobran cuando el número cambia de ancho, por ejemplo al pasar de `SCORE: 9` a `SCORE: 10`.

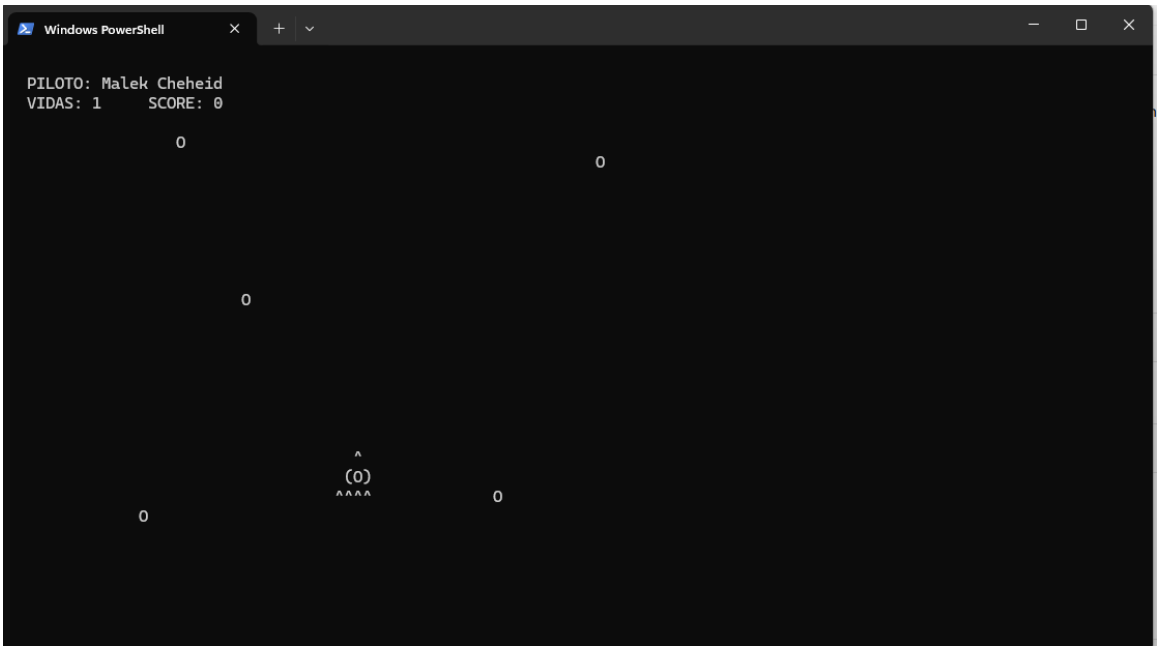
Tercero, sumo los puntos en la línea 353, dentro del bloque de colisión bala -> asteroide, apenas se destruye el asteroide impactado: `score += 10;`. Así cada acierto

vale 10 puntos y el marcador se actualiza en la misma vuelta del bucle.

Elegí esta funcionalidad porque se ve de inmediato en la consola, que es exactamente lo que pide la rúbrica para la captura de evidencia, y porque sirve de base para otras del anexo, como la dificultad progresiva o el disparo doble. Para verificarla compilé con `g++ -std=c++17 -Wall -Wextra src/main.cpp -o juego_naves.exe` y no hubo errores de compilación. Después ejecuté una partida y confirmé que el HUD muestra el nombre del piloto, las vidas restantes y el puntaje acumulado, como se ve en la Figura 1.

Figure 1

Ejecución de la Partida en Consola



Código completo `main.cpp`

```
/*
================================================================================
    Materia: Laboratorio de Programacion (LPR) - 5to Ano
    Institucion: E.E.S.T. N. 1 "Eduardo Ader" - Vicente Lopez
    Archivo: src/main.cpp
    Actividad 8: Introduccion a POO - Motor de Juego de Naves
                (Clases, Objetos, Encapsulamiento y Constructores)
    Docente: Mansilla Munoz York Elias
    Estudiante: Cheheid Malek

    Descripcion:
```

```
        Simulador de combate espacial en consola. El jugador maneja una nave
        que se desplaza por la pantalla, dispara proyectiles y pierde vidas
        cuando es alcanzada por los asteroides que descienden desde arriba.

    Controles:
        W / A / S / D  Mover la nave
        ESPACIO         Disparar

    Compilacion (MinGW):
        g++ src/main.cpp -o juego_naves.exe
================================================================================
*/

#include <iostream>   // Flujos de entrada y salida (cout, cin)
#include <windows.h>   // API de Windows: consola, Sleep, GetStdHandle
#include <conio.h>     // Entrada de teclado sin eco: _kbhit, _getch
#include <vector>      // Contenedor dinamico std::vector
#include <ctime>       // time() para la semilla del generador de numeros aleatorios
#include <cctype>      // tolower()

using namespace std;

/*
--------------------------------------------------------------------------------
    Funcion auxiliar: gotoxy
--------------------------------------------------------------------------------
    Coloca el cursor de la consola en la coordenada (x, y), donde el origen
    (0, 0) es la esquina superior izquierda. Se declara antes de las clases
    porque el cuerpo de los metodos pintar() y borrar() la invoca.

    Internamente se obtiene el handle de la salida estandar, se arma una
    estructura COORD con las coordenadas y se la pasa a SetConsoleCursorPosition.
--------------------------------------------------------------------------------
*/
void gotoxy(int x, int y) {
    HANDLE hCon = GetStdHandle(STD_OUTPUT_HANDLE);
    COORD dwPos;
    dwPos.X = x;
    dwPos.Y = y;
    SetConsoleCursorPosition(hCon, dwPos);
}

/*
--------------------------------------------------------------------------------
    Funcion auxiliar: ocultarCursor
--------------------------------------------------------------------------------
    Oculta el cursor de la consola para que el titilando no alterne con el
    dibujo del juego. SetConsoleCursorInfo recibe un puntero a la estructura
    CONSOLE_CURSOR_INFO, de ahi el uso del operador & (pasaje por referencia).
--------------------------------------------------------------------------------
*/
void ocultarCursor() {
    HANDLE hCon = GetStdHandle(STD_OUTPUT_HANDLE);
    CONSOLE_CURSOR_INFO cci;
    cci.dwSize = 2;
    cci.bVisible = FALSE;
    SetConsoleCursorInfo(hCon, &cci);
}

/*
================================================================================
    CLASE NAVE (el jugador)
================================================================================
```

```
    private:  los datos internos (x, y, vidas) quedan ocultos. Desde main() no se
              puede escribir directamente jugador.x = 99; solo se acceden a ellos
              mediante los metodos publicos. Eso es el encapsulamiento.

    public:   las acciones y los accesores. Son la unica "puerta controlada" que
              tiene el resto del programa para interactuar con el objeto.
================================================================================
*/
class Nave {
private:
    int x, y;     // Coordenadas de la nave en la consola
    int vidas;    // Cantidad de vidas restantes

public:
    /*
    ----------------------------------------------------------------------------
        CONSTRUCTOR: Nave(int _x, int _y, int _vidas)

        Es el metodo que se ejecuta al crear el objeto, por ejemplo:
            Nave jugador(35, 20, 3);

        Su trabajo es inicializar los atributos privados con los valores recibidos
        por parametro. El prefijo guion bajo (_x) se usa para distinguir los
        parametros de los atributos de la clase, ya que ambos se llaman igual.
        Sin este bloque, x, y y vidas quedarian con valores indeterminados.
    ----------------------------------------------------------------------------
    */
    Nave(int _x, int _y, int _vidas) {
        x = _x;
        y = _y;
        vidas = _vidas;
    }

    // Accesores: devuelven el valor de un atributo privado (solo lectura)
    int getX() { return x; }
    int getY() { return y; }
    int getVidas() { return vidas; }

    // Metodo de comportamiento: descuenta una vida
    void reducirVida() { vidas--; }

    /*
    ----------------------------------------------------------------------------
        Dibuja la nave en tres lineas: la punta superior, el cuerpo y la base.
        Cada gotoxy() reposiciona el cursor y el cout escribe el fragmento.
    ----------------------------------------------------------------------------
    */
    void pintar() {
        gotoxy(x, y);     cout << "  ^  ";
        gotoxy(x, y + 1); cout << " (O) ";
        gotoxy(x, y + 2); cout << "^^^^ ";
    }

    /*
    ----------------------------------------------------------------------------
        Borra la nave escribiendo espacios en las mismas posiciones donde la
        dibuja pintar(). Es indispensable antes de moverla para no dejar
        "estelas" de caracteres en la consola.
    ----------------------------------------------------------------------------
    */
    void borrar() {
        gotoxy(x, y);     cout << "     ";
        gotoxy(x, y + 1); cout << "     ";
        gotoxy(x, y + 2); cout << "     ";
```

```
    }

    /*
    ---------------------------------------------------------------------------
        Mueve la nave segun la tecla presionada.

        Cada comparacion combina dos condiciones gracias al operador logico AND
        (&&): no basta con que la tecla sea correcta, tambien se verifica que la
        nueva posicion no se salga del area de juego.
    ---------------------------------------------------------------------------
    */
    void mover(char tecla) {
        borrar();  // Primero se borra de la posicion anterior

        if (tecla == 'a' && x > 2)  x--;   // izquierda
        if (tecla == 'd' && x < 70) x++;   // derecha
        if (tecla == 'w' && y > 3)  y--;   // arriba
        if (tecla == 's' && y < 20) y++;   // abajo

        pintar(); // Luego se dibuja en la nueva posicion
    }
};

/*
===============================================================================
    CLASE ASTEROIDE
===============================================================================
    Enemigo que cae desde la parte superior de la pantalla. Al llegar al final
    reaparece arriba, en una columna aleatoria.
===============================================================================
*/
class Asteroide {
private:
    int x, y;

public:
    // Constructor: inicializa la posicion del asteroide
    Asteroide(int _x, int _y) { x = _x; y = _y; }

    int getX() { return x; }
    int getY() { return y; }

    void pintar() { gotoxy(x, y); cout << "O"; }
    void borrar() { gotoxy(x, y); cout << " "; }

    void mover() {
        borrar();
        y++;  // baja una fila por vuelta del bucle principal

        // Si llego al final del area de juego, reinicia arriba
        if (y > 23) {
            y = 4;
            x = (rand() % 70) + 2;  // columna aleatoria entre 2 y 71
        }

        pintar();
    }
};

/*
===============================================================================
    CLASE BALA
===============================================================================
    Proyectil que asciende. Se descarta cuando sale por arriba del area visible.
```

```
================================================================================
*/
class Bala {
private:
    int x, y;

public:
    Bala(int _x, int _y) { x = _x; y = _y; }

    int getX() { return x; }
    int getY() { return y; }

    void mover() {
        gotoxy(x, y); cout << " ";  // borra de la posicion anterior
        y--;                         // sube una fila
        gotoxy(x, y); cout << "|";  // dibuja en la nueva posicion
    }

    bool fueraDeRango() { return y < 4; }
};

/*
================================================================================
    PROGRAMA PRINCIPAL
================================================================================
*/
int main() {

    // Siembra el generador de numeros aleatorios con la hora actual, para que
    // las posiciones de los asteroides sean distintas en cada partida.
    srand(time(NULL));

    // Peticion del nombre del piloto: se muestra en pantalla y debe quedar
    // legible en la captura de evidencia.
    string nombrePiloto;
    cout << "Ingresa tu Nombre y Apellido (Se mostrara en la captura de pantalla): ";
    getline(cin, nombrePiloto);  // getline permite escribir nombre y apellido con espacios

    system("cls");     // limpia la pantalla
    ocultarCursor();   // oculta el cursor

    bool gameOver = false;
    int score = 0;  // Funcionalidad 1: Marcador de puntos

    // Se crea el objeto jugador: se ejecuta el constructor con (35, 20, 3)
    Nave jugador(35, 20, 3);

    // Vectores de punteros porque la cantidad de asteroides y balas cambia
    // dinamicamente durante la partida.
    vector<Asteroide*> asteroides;
    vector<Bala*> balas;

    // Carga inicial: 5 asteroides en posiciones aleatorias
    for (int i = 0; i < 5; i++) {
        asteroides.push_back(new Asteroide((rand() % 70) + 2, (rand() % 5) + 4));
    }

    jugador.pintar();

    /*
    --------------------------------------------------------------------------
        BUCLE PRINCIPAL
        Mientras queden vidas, repite: mostrar HUD, leer teclado, mover balas,
        mover asteroides y detectar colisiones.
```

```
--------------------------------------------------------------------------------
*/
while (!gameOver) {

    // HUD: nombre del piloto, vidas y puntaje
    gotoxy(2, 1);
    cout << "PILOTO: " << nombrePiloto << "  ";

    gotoxy(2, 2);
    cout << "VIDAS: " << jugador.getVidas() << "  ";

    gotoxy(15, 2);
    cout << "SCORE: " << score << "  ";

    // --- Entrada de teclado ---
    // _kbhit() indica si hay una tecla presionada pendiente de leer,
    // de ese modo el juego no se detiene esperando al usuario.
    if (_kbhit()) {
        // tolower() convierte la tecla a minuscula para que el Bloq Mayus
        // del teclado no impida jugar.
        char tecla = tolower(_getch());

        if (tecla == ' ') {
            // La bala nace dos columnas a la derecha y una fila arriba,
            // justo sobre el centro de la nave.
            balas.push_back(new Bala(jugador.getX() + 2, jugador.getY() - 1));
        } else {
            jugador.mover(tecla);
        }
    }

    // --- Proyectiles ---
    for (int i = 0; i < balas.size(); i++) {
        balas[i]->mover();

        // Si la bala salio del area de juego, se libera la memoria y se
        // quita del vector. El delete evita una fuga de memoria: sin el,
        // el espacio queda reservado pero inutilizable.
        if (balas[i]->fueraDeRango()) {
            gotoxy(balas[i]->getX(), balas[i]->getY());
            cout << " ";
            delete balas[i];
            balas.erase(balas.begin() + i);
        }
    }

    // --- Asteroides y colisiones ---
    for (int i = 0; i < asteroides.size(); i++) {
        asteroides[i]->mover();

        // COLISION ASTEROIDE -> JUGADOR
        // El asteroide tiene que caer dentro del rectangulo que ocupa la nave
        // (4 columnas de ancho por 3 filas de alto).
        if ((asteroides[i]->getX() >= jugador.getX() && asteroides[i]->getX() <=
            jugador.getX() + 4) &&
            (asteroides[i]->getY() >= jugador.getY() && asteroides[i]->getY() <=
                jugador.getY() + 2)) {

            jugador.borrar();
            jugador.reducirVida();

            // El asteroide impactado se reemplaza por uno nuevo arriba
            asteroides[i]->borrar();
            asteroides[i] = new Asteroide((rand() % 70) + 2, 4);
```

```
                jugador.pintar();

                // Si se agotaron las vidas, termina la partida
                if (jugador.getVidas() == 0) gameOver = true;
            }

            // COLISION BALA -> ASTEROIDE
            for (int j = 0; j < balas.size(); j++) {
                if (asteroides[i]->getX() == balas[j]->getX() &&
                    (asteroides[i]->getY() == balas[j]->getY() ||
                     asteroides[i]->getY() + 1 == balas[j]->getY())) {

                    // Se borra y libera la bala
                    gotoxy(balas[j]->getX(), balas[j]->getY());
                    cout << " ";
                    delete balas[j];
                    balas.erase(balas.begin() + j);

                    // Se borra y reemplaza el asteroide
                    asteroides[i]->borrar();
                    asteroides[i] = new Asteroide((rand() % 70) + 2, 4);

                    score += 10;  // Funcionalidad 1: sumar puntos por impacto

                    break;  // una bala no puede destruir dos asteroides
                }
            }
        }

        // Pausa breve para que el movimiento sea visible y no ocupe toda la CPU
        Sleep(30);
    }

    // --- Fin de la partida ---
    system("cls");
    gotoxy(30, 10);
    cout << "G A M E   O V E R... O sea... Perdiste!!";
    gotoxy(30, 12);
    system("pause");

    return 0;
}
```

Conclusión

Con esta actividad pasé de agrupar datos sueltos con `struct` a trabajar con objetos de verdad. La conclusión más clara para mí es que la diferencia de sintaxis entre las dos palabras es mínima —sólo el acceso por defecto—, pero lo que cambia en serio es quién manda sobre los datos.

Sobre el encapsulamiento me quedó demostrado que proteger `vidas` no es un capricho de la materia. Al dejar una única vía de escritura, `reducirVida()`, y una de sólo lectura, `getVidas()`, el objeto se cuida solo y yo desde `main()` pierdo —y dejo de necesitar— la posibilidad de romperlo. Sobre los constructores entendí que su función es que un objeto nazca con valores definidos; sin ellos estaría leyendo basura de memoria.

El motor que entrega la actividad compila y corre con TDM-GCC 10.3 en modo C++17, e implementa las tres clases, el bucle principal, las dos detecciones de colisiones —bala con asteroide y asteroide con nave— y la memoria dinámica con `new` y `delete` sobre contenedores `std::vector` de punteros. Yo verifiqué que todo eso funcione en mi máquina y le sumé la funcionalidad obligatoria de marcador de puntos, que se ve en el HUD y en la captura de evidencia.

Al revisar el código encontré tres cosas que quiero corregir después. Falta un `delete` del asteroide que se reemplaza al impactar, para que la memoria quede tan limpia como ya la queda con las balas. Conviene sumar `#include <string>`, porque hoy depende de inclusiones transitivas. Y en los tres bucles que recorren un `std::vector` el índice está declarado como `int`, con lo que quedan tres advertencias de comparación entre signo distinto que se arreglan pasándolo a `size_t`. Ninguna de las tres rompe el juego, pero me parece que anotarlas también forma parte del trabajo.

Referencias

Cheheid, M. (2026a). *Juego de naves en consola* (v1.0.0) [Repositorio de código fuente]. GitHub. https://github.com/malekcheheid-art/juegodenaves.git

Cheheid, M. (2026b). *Juego de naves en consola* (v1.0.0) [Repositorio de código fuente]. GitLab. https://gitlab.com/malekcheheid-art/juegodenaves.git

cplusplus.com. (s. f.). *Classes (I)*. Recuperado el 21 de octubre de 2026, de https://cplusplus.com/doc/tutorial/classes/

GeeksforGeeks. (s. f.). *Access modifiers in C++*. Recuperado el 21 de octubre de 2026, de https://www.geeksforgeeks.org/access-modifiers-in-c/

Git. (s. f.). *Git documentation*. Recuperado el 21 de octubre de 2026, de https://git-scm.com/doc

Microsoft. (2022, 7 de febrero). *Constructores (C++)*. Microsoft Learn. https://learn.microsoft.com/es-es/cpp/cpp/constructors-cpp